Validación de formularios con la reactividad de Vue

Clase 02 · De ref y computed al formulario de registro real (bloque 3 de user-register)

Los archivos que estudiamos

- `src/composables/useRegister.ts` — lo propio del registro: los valores, las reglas del cliente, la llamada a `register` y qué significa el 409.
- `src/composables/useForm.ts` — lo que comparte cualquier formulario: si ya se envió, `loading`, los pasos del envío y **cuándo** se ve cada error.
- `src/composables/useFormErrors.ts` — **qué** está mal: reparte los errores del back por campo y los junta con los del cliente. Lo usa `useForm` por dentro.
- `src/components/auth/RegisterForm.vue` — pinta los campos, sus errores y los errores generales arriba.
- `src/__tests__/RegisterForm.spec.ts`, `useForm.spec.ts`, `useFormErrors.spec.ts` y `e2e/register.spec.ts` — los tests unitarios y end-to-end.

Las referencias van como archivo:línea, por ejemplo `useRegister.ts:38`. Los números son los del código en el momento de escribir esta clase.

Qué cambia en el bloque, en una frase por punto

- Antes había un solo mensaje `error` debajo del formulario y un `passwordMismatch` que deshabilitaba el botón.
- Ahora cada campo tiene sus propios mensajes (del cliente y del back), los errores que no son de ningún campo van arriba, y el botón solo se deshabilita mientras se envía.
- Las reglas del cliente (obligatorio, email válido, contraseña de 8, confirmación igual) son un `computed` llamado `clientErrors`.
- Los errores no se ven hasta el primer envío; después, los del cliente se actualizan en vivo mientras escribes.
- El 409 (`email_already_registered`) marca el campo email con "Ya existe una cuenta registrada con ese email.". Para ir al login está el enlace "Inicia sesión" del pie del formulario; no hay un segundo enlace junto al error.
- Tras el bloque se partió la lógica en tres capas: `useRegister` (lo del registro) → `useForm` (el ciclo de vida de cualquier formulario) → `useFormErrors` (solo los errores). La clase describe el código ya con esas tres capas.

Cómo está organizada esta clase

Cada tema sigue el mismo recorrido: 1) el concepto con palabras, 2) un ejemplo sencillo sin nada del proyecto, 3) un recuadro amarillo que señala dónde aparece en el código real. Los recuadros azules marcan la idea que conviene recordar.

Las partes van de menos a más: primero las piezas reactivas de Vue (`ref`, `computed`, `watch`), después cómo y cuándo se ejecutan (la parte más importante para entender el bloque), luego composables y plantillas, la accesibilidad del formulario, y al final un recorrido completo por el código y por los tests.

Índice

Parte I · Piezas reactivas — 1. `ref` y `.value` · 2. `shallowRef` · 3. `computed`: valores derivados · 4. `watch`: reaccionar con efectos · 5. `readonly`

Parte II · Cuándo se ejecuta cada cosa — 6. Dependencias que se apuntan al ejecutar · 7. Ramas: solo cuenta lo que se leyó · 8. Avisar no es ejecutar: sucio y perezoso · 9. El render se encola · 10. La cadena completa

Parte III · Composables — 11. Qué es un composable · 12. Composables en capas · 13. `MaybeRefOrGetter` y `toValue` · 14. Tipos genéricos para los errores · 15. Pasar funciones: inversión de control

Parte IV · Plantilla y accesibilidad — 16. `v-if`, `v-for` y `:key` · 17. Props enlazadas y atributos que desaparecen · 18. Emits: eventos hacia arriba · 19. Atributos fallthrough e `inputProps` · 20. Formularios accesibles

Parte V · Recorrido por el código — 21. Las reglas del cliente · 22. `submit()` paso a paso · 23. Qué está mal y cuándo se ve · 24. Cronología de una sesión real · 25. `RegisterForm.vue`

Parte VI · Tests — 26. Tests de componente con Vue Test Utils · 27. Tests de composables con `effectScope` · 28. Tests e2e con Playwright

Anexo — Chuleta

1. ref y .value: una caja que avisa

Una variable normal de JavaScript no avisa a nadie cuando cambia. Vue necesita saber cuándo cambia un dato para volver a pintar lo que depende de él, así que lo metes en una "caja" reactiva con `ref()`. El valor está dentro, en `.value`.

```
import { ref } from 'vue'

const contador = ref(0)        // una caja con un 0 dentro

console.log(contador.value)    // 0   (leer)
contador.value++               // 1   (escribir: Vue se entera)
```

Dos operaciones importan: **leer** `.value`, que le dice a Vue "quien me está leyendo depende de mí", y **escribir** `.value`, que le dice a Vue "avisa a quien dependa de mí". Toda la Parte II se construye sobre estas dos frases.

En la plantilla no se escribe `.value`: Vue lo desenvuelve solo. `{{ contador }}` en la plantilla equivale a `contador.value` en el script.

En el código

`useRegister.ts:29-33`: cada campo del formulario (`name`, `email`, `password`, `passwordConfirmation`, `accountToken`) es un `ref`. En `RegisterForm.vue`, `v-model="email"` lee esa caja para pintar el input y la escribe con cada tecla.

2. shallowRef: reactiva solo al reemplazar

`ref()` con un objeto dentro lo hace reactivo "en profundidad": Vue envuelve el objeto en un proxy y detecta también cambios en sus propiedades internas. `shallowRef()` solo detecta que reemplaces el `.value` entero; cambiar algo por dentro no avisa a nadie.

```
const profundo = ref({ n: 1 })
profundo.value.n = 2      // avisa (Vue vigila las propiedades)

const plano = shallowRef({ n: 1 })
plano.value.n = 2         // NO avisa: nadie se entera
plano.value = { n: 2 }    // avisa: reemplazaste la caja entera
```

Con un valor primitivo (un booleano, un número, un string) no hay "por dentro", así que `ref` y `shallowRef` se comportan igual. Elegir `shallowRef` comunica la intención: "este valor solo se sustituye entero" y evita crear proxies que no se necesitan.

En el código

`useFormErrors.ts:65-66`: `backFieldErrors` y `generalErrors` son `shallowRef`. Por eso `setFieldError` (`useFormErrors.ts:123-127`) no hace `backFieldErrors.value[field] = ...`, sino que copia el objeto con spread, cambia la copia y la asigna entera. Si mutara por dentro, la pantalla no se actualizaría.

`useForm.ts:52-53`: `submitted` y `loading` también son `shallowRef`. Son booleanos, así que da igual en la práctica; el `shallowRef` comunica que solo se sustituyen.

`useRegister.ts` no crea ningún `shallowRef` propio: sus campos son `ref` de texto y el resto del estado lo guardan `useForm` y `useFormErrors`.

3. computed: un valor derivado y cacheado

Un `computed` es un valor que se **calcula a partir de otros** valores reactivos. Le das una función; Vue la ejecuta, guarda el resultado (caché) y te lo devuelve en `.value`. Cuando cambia algo de lo que leyó, el resultado guardado deja de valer y se recalcula la próxima vez que lo pidas.

```
const nombre = ref('Ellen')
const apellido = ref('Ripley')

const completo = computed(() => `${nombre.value} ${apellido.value}`)

completo.value     // 'Ellen Ripley'
completo.value     // 'Ellen Ripley' (de la caché, no recalcula)
apellido.value = 'R.'
completo.value     // 'Ellen R.'     (recalcula porque apellido cambió)
```

Reglas de buen uso: la función de un `computed` **solo calcula y devuelve**. No escribe en otros refs, no llama a la API, no toca el DOM. Si necesitas "hacer algo" cuando un valor cambia, eso es un `watch` (tema 4).

> **Idea clave: derivar en vez de sincronizar**
>
> Si un dato se puede calcular a partir de otros, no lo guardes en un `ref` aparte que tengas que mantener al día a mano: hazlo `computed`. Así es imposible que se desincronice.

> **En el código**
>
> `useRegister.ts:35-66`: `clientErrors` es un `computed` que, a partir de los cinco campos, devuelve un objeto con los errores de cada campo. No guarda nada ni muestra nada: solo responde "con los valores actuales, estos son los errores".
>
> Sustituye al antiguo `passwordMismatch`, que también era un `computed`, pero solo cubría una regla.

4. watch: reaccionar con un efecto

`watch(fuente, callback)` ejecuta el callback **cada vez que cambia** la fuente. Es para efectos: cosas que hay que hacer, no valores que hay que calcular.

```
const busqueda = ref('')

watch(busqueda, (nuevo, anterior) => {
  console.log(`cambió de "${anterior}" a "${nuevo}"`)
  // aquí sí puedes escribir en otros refs, llamar a la API...
})
```

Diferencias con `computed`:

	`computed`	`watch`
Para qué	Calcular un valor	Hacer algo cuando algo cambia
Devuelve	Un valor en `.value`	Nada útil (una función para pararlo)
Cuándo corre	Solo si alguien lo lee (perezoso)	Siempre que la fuente cambie
Efectos dentro	No	Sí, para eso existe

Por defecto el callback de un `watch` no corre en el mismo instante de la escritura: Vue lo encola y lo ejecuta justo antes de actualizar la pantalla (opción `flush: 'pre'`). Vuelve a salir en el tema 9.

En el código

`useFormErrors.ts:91-98`: un `watch` por campo borra el error del back de ese campo en cuanto editas su valor. Es un efecto ("cuando el email cambie, olvida lo que dijo el back sobre él"), no un cálculo: el front no sabe volver a aplicar la regla del back, solo sabe que el valor que la provocó ya cambió. Por eso no puede ser un `computed`.

`useRegister.ts` y `useForm.ts` no tienen ningún `watch`: lo suyo es derivado o lo delegan en `useFormErrors`. Incluso el error del 409 en el email se borra con este mismo `watch`, porque para `useFormErrors` es un error del back más.

Que este `watch` siga en `useFormErrors` y no en `useForm` es a propósito: no decide "cuándo se ve" un error, sino que el error ya no es verdad (el back juzgó un valor que ya no existe).

5. readonly: dejar mirar, no tocar

`readonly(algo)` devuelve una vista de solo lectura de un ref u objeto reactivo. Quien la recibe puede leer el valor (y sigue siendo reactivo), pero si intenta escribir, TypeScript da error y Vue avisa en consola en desarrollo.

```
const interno = shallowRef<string[]>([])
const publico = readonly(interno)

publico.value            // [], y se actualiza cuando cambia interno
publico.value = ['x']    // ✗ error de tipos + aviso de Vue
interno.value = ['x']    // ok: solo el dueño escribe
```

En el código

`useFormErrors.ts:135` devuelve `generalErrors: readonly(generalErrors)`. `useForm` y `useRegister` lo pasan tal cual y `RegisterForm` lo lee para pintar los errores de arriba, pero solo `useFormErrors` lo escribe: lo llena al recibir un error del back y lo vacía con `clearGeneralErrors()`, que `useForm` llama al empezar cada envío.

`useForm.ts:101` hace lo mismo con `loading: readonly(loading)`: el componente lo lee para deshabilitar el botón, pero solo `submit` lo cambia.

`fieldErrors` no necesita `readonly`: es un `computed`, y un `computed` sin setter ya es de solo lectura.

Es la regla "el componente pinta, el composable decide" de `doc/conventions.md`, reforzada por los tipos.

Parte II · Cuándo se ejecuta cada cosa

Esta parte responde a la pregunta que más se presta a confusión en el bloque: **¿cuándo se ejecuta la función de `clientErrors`?** La respuesta sale de cuatro reglas que van una detrás de otra.

6. Las dependencias se apuntan al ejecutar

Un `computed` no declara de qué depende. Vue lo descubre **mientras la función se ejecuta**: cada `.value` que la función lee queda apuntado como dependencia. Antes de la primera ejecución, el computed no tiene ninguna dependencia apuntada, así que nadie puede avisarle de nada.

```
const a = ref(1)
const b = ref(2)
const suma = computed(() => a.value + b.value)

// Ahora mismo suma NO depende de nada: nunca se ha ejecutado.
suma.value     // se ejecuta → lee a y b → apunta "dependo de a y b"
```

Lo mismo vale para el render de un componente y para `watchEffect`: todos son "observadores" que apuntan lo que leen mientras corren.

7. Ramas: solo cuenta lo que se leyó

Como las dependencias se apuntan al ejecutar, si un `if` impide que se llegue a leer algo, ese algo **no** queda apuntado en esa ejecución. Y Vue vuelve a apuntar desde cero en cada ejecución, así que las dependencias pueden cambiar de una vez a otra.

```
const mostrar = ref(false)
const detalle = ref('hola')

const texto = computed(() => (mostrar.value ? detalle.value : ''))

texto.value          // lee mostrar (false) y ya: depende SOLO de mostrar
detalle.value = 'x'  // texto no se entera: no depende de detalle

mostrar.value = true
texto.value          // ahora lee mostrar Y detalle: depende de los dos
detalle.value = 'y'  // ahora sí: texto queda desactualizado
```

En el código

`useForm.ts:63` no le pasa a `useFormErrors` el `computed` `clientErrors` tal cual, sino una función (un getter) con una rama: si ya se envió, devuelve los errores del cliente; si no, un objeto vacío.

```
clientErrors: () => (submitted.value ? (toValue(clientErrors) ?? {}) : {}),
```

Ese getter no es un `computed`: es una función normal. Se ejecuta cuando `fieldErrors` (`useFormErrors.ts:75`) hace `toValue(clientErrors)`, y como corre **dentro** del computed `fieldErrors`, todo lo que lee queda apuntado como dependencia de `fieldErrors`. Antes del primer envío, `fieldErrors` depende de `submitted` y de `backFieldErrors`, pero **no** de `clientErrors`.

Fíjate en el reparto: la rama ("¿se ve ya?") la escribe `useForm`; `useFormErrors` solo junta lo que recibe, sin saber que hay una condición detrás.

8. Avisar no es ejecutar: sucio y perezoso

Cuando escribes un ref, Vue avisa a quien depende de él. Pero qué hace cada uno con el aviso es distinto:

- Un **computed** avisado solo se marca como "sucio" (su caché ya no vale) y pasa el aviso a quien dependa de él. **No se recalcula.** Se recalculará la próxima vez que alguien lea su `.value`. A esto se le llama ser perezoso (lazy).
- Un **efecto** avisado sí se ejecuta. Efectos son el render de un componente, `watch` y `watchEffect`.

```
let veces = 0
const a = ref(1)
const doble = computed(() => { veces++; return a.value * 2 })

a.value = 5
a.value = 6
a.value = 7        // veces sigue en 0: nadie ha leído doble

doble.value        // 14 → veces = 1 (una sola ejecución)
doble.value        // 14 → veces = 1 (caché)
```

> **Idea clave: un computed se ejecuta solo si está sucio Y alguien lo lee**
>
> Si nadie lo lee, da igual cuántas veces cambien sus dependencias: no se ejecuta. Si lo leen pero no está sucio, devuelve la caché.

Un detalle de Vue 3.4 o superior (el proyecto usa 3.5): si un computed se recalcula y devuelve un valor idéntico al anterior (`Object.is`), quien depende de él no se vuelve a ejecutar. `clientErrors` devuelve un objeto nuevo en cada ejecución, así que para Vue siempre "ha cambiado".

9. El render se encola

Cuando el render de un componente recibe un aviso, no se repinta en ese mismo instante. Vue lo **encola** y lo ejecuta cuando termina el código síncrono que está corriendo (en una microtarea). Así, diez escrituras seguidas producen un solo render. Los `watch` con el `flush` por defecto también se encolan, y corren antes del render.

```
contador.value = 1
contador.value = 2
contador.value = 3
// La pantalla todavía muestra el valor viejo: el render está encolado.

await nextTick()
// Ahora sí se ha repintado, una sola vez, con el 3.
```

Consecuencia práctica: si tras escribir un ref lees un computed en la línea siguiente, esa lectura ocurre antes de que el render encolado tenga oportunidad de leerlo.

En el código

`useForm.ts:74-77`: `submit` pone `submitted` a `true` (avisa a `fieldErrors`, que avisa al render, que se encola) y, sin esperar a nada, la línea 77 llama a `hasClientErrors()`, que lee `clientErrors` (línea 67). La línea 77 gana: es la primera vez que se ejecuta `clientErrors`.

```
submitted.value = true         // render encolado
clearGeneralErrors()

if (hasClientErrors()) return null   // ← 1ª ejecución de clientErrors
```

10. La cadena completa

Juntando las cuatro reglas, la función de `clientErrors` puede llegar a ejecutarse por dos caminos: la lee directamente `hasClientErrors()` al enviar (`useForm.ts:77`), o la lee el render a través de `fieldErrors`. El segundo camino solo existe cuando está montada esta cadena de suscripciones:

`name`, `email`, `password`, ...	→	`clientErrors`	→	`fieldErrors`	→	render del componente
(refs, `useRegister`)		(computed, `useRegister`)		(computed, `useFormErrors`)		(efecto)

- `clientErrors` depende de los cinco refs: los apuntó en su primera ejecución (al enviar).
- `fieldErrors` depende de `clientErrors`: lo apunta solo cuando el getter de `useForm` se ejecuta con `submitted` a `true` (tema 7).
- El render depende de `fieldErrors` porque la plantilla lo lee.

El getter de `useForm` no aparece como eslabón propio: al no ser un `computed`, no tiene caché ni suscripciones; sus lecturas se apuntan a nombre de `fieldErrors`. Las tres capas (`useRegister`, `useForm`, `useFormErrors`) existen en el código, pero para la reactividad la cadena es la misma de cuatro piezas.

Antes del primer envío el eslabón del medio no existe: escribir en `name` re-renderiza (la plantilla muestra `name` por el `v-model`), pero `fieldErrors` no está sucio y devuelve la caché. Nadie toca `clientErrors`.

Después del primer envío, una tecla en "Nombre" hace esto:

- 1. `v-model` escribe `name.value`.
- 2. `clientErrors` se marca sucio (no se ejecuta).
- 3. El aviso sube: `fieldErrors` se marca sucio.
- 4. El aviso llega al render, que es un efecto: se encola.
- 5. Corre el render, lee `fieldErrors` (sucio → se recalcula), que lee `clientErrors` (sucio → se ejecuta).

Idea clave: lo que cambió al enviar no es clientErrors, es quién lo lee

Si la plantilla no usara `fieldErrors`, `clientErrors` solo se ejecutaría en `hasClientErrors()`, una vez por envío, por mucho que escribieras. Esto se puede comprobar con un breakpoint en `useRegister.ts:36`: no para al cargar ni al escribir antes de enviar; para por primera vez al pulsar "Crear cuenta" (con `submit` y `hasClientErrors` en la pila) y desde entonces con cada tecla (con el render en la pila).

11. Qué es un composable

Un composable es una función normal que usa la reactividad de Vue (`ref`, `computed`, `watch`...) y devuelve lo que quiere compartir. Por convención su nombre empieza por `use`. Sirve para sacar del componente la lógica con estado, de forma que el componente se quede solo con pintar.

```
// useContador.ts
export function useContador() {
  const n = ref(0)
  const par = computed(() => n.value % 2 === 0)
  function sumar() { n.value++ }
  return { n, par, sumar }
}

// En un componente
const { n, par, sumar } = useContador()
```

Cada llamada crea su propio estado: dos componentes que llaman a `useContador()` tienen contadores distintos. (Si quisieras un estado compartido por toda la app, eso sería una store de Pinia.)

Los `watch` y `computed` creados dentro de un composable llamado durante el `setup` de un componente quedan ligados a ese componente: Vue los para solo cuando el componente se desmonta.

En el código

`RegisterForm.vue:17-27` llama a `useRegister()` y desestructura lo que necesita pintar. Toda la lógica (reglas, cuándo mostrar errores, qué hacer con cada error del back) vive en los composables.

12. Composables en capas

Los composables se pueden componer: uno llama a otro y le pasa sus propios refs. El de dentro resuelve una parte genérica y el de fuera añade lo específico. Se puede repetir: tres capas, cada una con una sola responsabilidad.

```
function useBusqueda() {
  const texto = ref('')
  const { resultados } = useFiltro({ fuente: texto })   // le pasa su ref
  return { texto, resultados }
}
```

En el código

El formulario de registro usa tres capas, de lo concreto a lo genérico:

```
RegisterForm.vue  →  useRegister  →  useForm  →  useFormErrors
   (pinta)          (registro)     (ciclo de vida)  (qué está mal)
```

`useRegister.ts:68-83` llama a `useForm` con los cinco refs, las reglas, la llamada a `register` y el 409. Recibe `loading`, `fieldErrors`, `generalErrors` y `submit`, y se los devuelve al componente.

`useForm.ts:55-64` llama a `useFormErrors` con los campos y sus errores del cliente ya filtrados por "¿se ha enviado?". Se queda con `setFromApiError`, `setFieldError` y `clearGeneralErrors` para usarlos dentro de `submit`, y solo devuelve hacia fuera lo que el componente necesita.

El reparto de responsabilidades:

Capa	Sabe...	No sabe...
`useRegister`	Qué campos tiene el registro, sus reglas, que se llama a `register` y que el 409 es un error del email.	Cómo se envía un formulario ni cómo se reparten los errores.
`useForm`	Si ya se envió, `loading`, el orden de los pasos del envío y cuándo se ve cada error.	Qué es un registro ni qué significa ningún código concreto.
`useFormErrors`	Repartir los `violations` de un 422, los mensajes genéricos en español y juntar errores del cliente y del back.	Si el formulario se envió ni cuándo debe verse nada.

Idea clave: una capa solo habla con la de al lado

`useRegister` no importa nada de `useFormErrors`: ni siquiera el tipo `FieldErrors`, que le llega reexportado por `useForm` (tema 14). Si mañana `useForm` organizara los errores de otra forma, `useRegister` no se enteraría.

13. MaybeRefOrGetter y toValue

Un composable reutilizable a veces quiere aceptar un valor "venga como venga": un valor plano, un ref o una función que lo devuelve. El tipo `MaybeRefOrGetter<T>` describe esas tres formas, y `toValue()` las convierte todas en el valor de dentro.

```
import { toValue, type MaybeRefOrGetter } from 'vue'

function saludo(nombre: MaybeRefOrGetter<string>) {
  return computed(() => `Hola, ${toValue(nombre)}`)
}

saludo('Ana')               // valor plano
saludo(ref('Ana'))          // ref
saludo(() => user.name)     // getter
```

Detalle importante: si `toValue` se llama dentro de un `computed` y lo que recibe es un ref o un getter, la lectura queda apuntada como dependencia. Por eso el computed sigue siendo reactivo.

En el código

Aquí se usan dos de las tres formas, y por eso compensa aceptar `MaybeRefOrGetter`:

`useRegister` le pasa a `useForm` un `computed` (un ref). `useForm.ts:39` lo declara como `MaybeRefOrGetter` y lo lee con `toValue` en `hasClientErrors` (línea 67) y dentro del getter (línea 63).

`useForm` le pasa a `useFormErrors` un **getter** (`() => ...`, línea 63). `useFormErrors.ts:26` también acepta `MaybeRefOrGetter`, y `useFormErrors.ts:75` lo lee con `toValue(clientErrors)` dentro del computed `fieldErrors`. `toValue` llama al getter, y esa llamada dentro del computed es la que hace que `fieldErrors` dependa de `submitted` y (tras enviar) de `clientErrors` (temas 7 y 10).

Si `useFormErrors` solo aceptara un `Ref`, `useForm` tendría que crear un `computed` intermedio solo para envolver la condición. El getter hace lo mismo sin caché propia.

14. Tipos genéricos para los errores

Esto es TypeScript (repasa la clase 01, temas 3, 13 y 14), aplicado aquí. `Record<K, V>` es un objeto con claves `K` y valores `V`; `Partial<T>` hace opcionales todas las propiedades. Juntos describen "un objeto que puede tener algunas de estas claves".

```
type Campo = 'nombre' | 'email'
type Errores = Partial<Record<Campo, readonly string[]>>

const e: Errores = {}                       // ok: ninguna clave
e.email = ['Introduce un email válido.']    // ok
e.telefono = ['...']                        // ✗ 'telefono' no es un Campo
```

En el código

`useFormErrors.ts:18-20` define `FieldErrors<Field>` así, con `Field` como parámetro genérico. `useRegister.ts:16-17` define `RegisterField` como la unión de los cinco nombres y la usa para tipar `clientErrors`.

`useForm<Field, Result>` (`useForm.ts:46`) tiene dos genéricos y `useRegister` no escribe ninguno: TypeScript los **infiere**. `Field` sale de las claves de `fields` (los cinco nombres) y `Result` de lo que devuelve `action` (la respuesta de `register`). Por eso `submit()` devuelve el usuario creado o `null`, y si en el 409 escribes `setFieldError('emial', ...)`, el error salta al compilar.

Cada campo guarda una lista de mensajes (`readonly string[]`) porque un campo puede tener un error del cliente y otro del back a la vez.

Reexportar un tipo

Un módulo puede volver a exportar un tipo que importó de otro: `export type { X }`. Quien lo use lo importa del módulo que reexporta, sin conocer el original.

```
// b.ts
import type { X } from './a'
export type { X }          // b ofrece X como parte de su API

// c.ts
import type { X } from './b'   // c no sabe que X nace en a.ts
```

En el código

`useForm.ts:21-22` reexporta `FieldErrors`, porque forma parte de su API: hace falta para tipar `clientErrors`. Así `useRegister.ts:14` importa el tipo de `useForm`.

El proyecto evita los "barrels" (archivos índice que solo reexportan) siguiendo la guía de rendimiento de Vite, pero esto no lo es: `useForm.ts` es un módulo de verdad, y un `export type` se borra al compilar (clase 01, tema 1), así que no genera ningún import en ejecución.

15. Pasar funciones: inversión de control

Normalmente tu código llama a una librería. Con **inversión de control** es al revés: le pasas funciones y es ella quien decide cuándo llamarlas. El patrón **Template Method** es un caso concreto: los pasos fijos se escriben una sola vez y quien lo usa solo rellena los huecos que cambian.

```
function conReintento(tarea: () => Promise<string>) {
  // Pasos fijos: intentar, y si falla, un segundo intento.
  return tarea().catch(() => tarea())
}

// Quien lo usa solo aporta el hueco: QUÉ tarea.
conReintento(() => fetch('/api/hola').then((r) => r.text()))
```

Las librerías de formularios de Vue funcionan así: vee-validate tiene `handleSubmit(fn)` y TanStack Form un `onSubmit`. El formulario lleva el ciclo de envío y tú le das la función que llama a la API.

En el código

`useForm` recibe dos huecos (`useForm.ts:40-43`):

`action`: la función que llama a la API. En `useRegister.ts:71-77`, llama a `register` con los cuatro valores. `useForm` decide cuándo llamarla (solo si pasan las reglas del cliente) y qué hacer con lo que pase.

`errorHandlers`: un mapa `código → función` para los errores que tienen un significado propio en ese formulario. Su tipo (`useForm.ts:28-33`) es `Partial<Record<ApiErrorCode, ...>>`: las claves solo pueden ser códigos de error reales, y TypeScript avisa si escribes mal uno. En `useRegister.ts:78-82` tiene una sola entrada, `email_already_registered`.

Cada función del mapa recibe el error y unas utilidades (`{ setFieldError }`). Las recibe como argumento porque `useRegister` define el mapa **antes** de tener lo que devuelve `useForm`.

Idea clave: el mapa por código impide decidir por status

Con un callback libre (`onError(error)`), cada formulario podría mirar `error.status` o el texto del mensaje, y la regla del proyecto es decidir siempre por `error.code`. Con el mapa, la única forma de "engancharse" a un error es por su código.

Lo que se gana: el orden de los pasos (cortar por reglas, `loading`, `try/finally`, relanzar lo que no sea `ApiError`) se escribe una sola vez y ningún formulario puede saltarse uno. Lo que cuesta: para seguir el flujo de un envío hay que leer dos archivos. Con un solo formulario se podría haber esperado (YAGNI); se hizo ya porque login y reenvío vienen detrás.

16. v-if, v-for y :key

`v-if="condición"` pone o quita un elemento del DOM según la condición. `v-for="x in lista"` repite el elemento una vez por elemento de la lista. Con `v-for` hay que dar un `:key` único y estable, para que Vue sepa qué elemento es cuál cuando la lista cambia y no reutilice el nodo equivocado.

```
<ul v-if="tareas.length > 0">
  <li v-for="tarea in tareas" :key="tarea.id">{{ tarea.texto }}</li>
</ul>
```

En el código

`RegisterForm.vue:57-68`: los errores generales salen arriba del formulario con `v-if` sobre el contenedor y un `<Message>` por mensaje con `v-for`. El `:key` es el propio mensaje: dentro de una misma lista no se repiten.

Debajo de cada campo se repite el mismo patrón: un `<div v-if="fieldErrors.name">` con un `<Message>` por cada mensaje de ese campo.

17. Props enlazadas y atributos que desaparecen

`:prop="expresión"` (abreviatura de `v-bind:prop`) pasa el resultado de una expresión en vez de un texto fijo. Si el destino es un atributo HTML normal y la expresión vale `undefined` o `null`, Vue **quita el atributo** del DOM en lugar de escribir "undefined".

```
<input :title="ayuda" />
<!-- ayuda = 'Escribe tu nombre'  →  <input title="Escribe tu nombre"> -->
<!-- ayuda = undefined            →  <input>   (sin atributo)          -->
```

En el código

`RegisterForm.vue:113`: `:invalid="fieldErrors.email !== undefined"` pasa un booleano a la prop `invalid` de PrimeVue, que pinta el borde rojo y pone `aria-invalid` en el input.

`RegisterForm.vue:38-40`: `describedBy(field)` devuelve el id del contenedor de errores si el campo tiene errores, y `undefined` si no. Así `aria-describedby` solo existe cuando hay algo que describir.

`RegisterForm.vue:29-36`: los ids están en un objeto `errorIds` con `as const`, que hace que TypeScript conozca las claves exactas; `keyof typeof errorIds` limita `describedBy` a esos cinco nombres.

18. Emits: eventos hacia arriba

Un hijo no cambia el estado del padre directamente: **emite un evento** y el padre decide qué hacer. Es el patrón "props hacia abajo, eventos hacia arriba". Con `defineEmits` y tipos, TypeScript comprueba el nombre del evento y sus argumentos.

```
<!-- Hijo -->
const emit = defineEmits<{ cerrar: [] }>()
<button @click="emit('cerrar')">X</button>

<!-- Padre -->
<Hijo @cerrar="abierto = false" />
```

En el código

`RegisterForm.vue:240-249`: el enlace "Inicia sesión" del pie del formulario hace `emit('switchToLogin')`. `RegisterForm` no sabe cómo se cambia de pestaña; lo sabe `AuthDialog.vue`, que escucha ese evento y cambia a la pestaña de login.

Es el único camino al login desde el formulario: tras un 409, el usuario ve el error en el email y usa este mismo enlace.

Es un `<button type="button">` y no un enlace: no navega a ninguna URL, ejecuta una acción. El `type="button"` evita que, al estar dentro del `<form>`, lo envíe.

El otro evento, `registered`, lo emite `handleSubmit` (`RegisterForm.vue:42-48`) cuando `submit()` devuelve el usuario creado.

19. Atributos fallthrough e inputProps

Si pasas a un componente un atributo que no ha declarado como prop (por ejemplo `aria-describedby`), Vue lo deja caer (**fallthrough**) sobre el elemento raíz del componente. Funciona bien cuando la raíz es el `<input>`, pero no cuando el input está envuelto en un `<div>`.

```
<!-- Componente MiCampo: la raíz es un div -->
<div class="envoltorio"><input /></div>

<!-- Uso -->
<MiCampo aria-describedby="ayuda" />
<!-- Resultado: <div class="envoltorio" aria-describedby="ayuda"><input></div> -->
<!-- El lector de pantalla lee el input... y la descripción no está en él. -->
```

En el código

`InputText` de PrimeVue renderiza el `<input>` como raíz, así que `:aria-describedby` le llega directo (`RegisterForm.vue:83`).

`Password` envuelve el input (para el botón de mostrar/ocultar), así que el atributo caería en el envoltorio. PrimeVue ofrece la prop `inputProps` para pasar atributos al `<input>` interno; por eso `RegisterForm.vue:143` usa `:input-props="{ 'aria-describedby': ... }"`. El comentario de la línea 139 lo deja escrito para que nadie lo "simplifique".

20. Formularios accesibles

Varias decisiones del bloque existen para que el formulario funcione bien con lector de pantalla y teclado, no solo con la vista:

Pieza	Qué hace	Dónde
`novalidate` en el `<form>`	Desactiva las burbujas de validación del navegador (estilo propio y en el idioma del navegador). Los mensajes los pone la app.	`RegisterForm.vue:54`

Pieza	Qué hace	Dónde
`required` en los inputs	Se mantiene: aunque ya no bloquee el envío, sigue diciendo al lector de pantalla que el campo es obligatorio.	cada input
`aria-invalid`	Anuncia "no válido" al enfocar el campo. Lo pone la prop `invalid` de PrimeVue.	`:invalid=...`
`aria-describedby`	Une el input con su mensaje: al enfocar el campo, el lector lee también el error.	`describedBy()`
Botón nunca deshabilitado por validación	Un botón deshabilitado no dice por qué lo está, y algunos lectores no lo anuncian. Mejor dejar pulsar y explicar el error. Solo se deshabilita mientras `loading`.	`RegisterForm.vue:235`
Errores generales arriba	Se ven y se leen antes que el resto del formulario.	`RegisterForm.vue:57`

21. Las reglas del cliente

`clientErrors` (`useRegister.ts:35-66`) empieza con un objeto vacío y va añadiendo un mensaje por cada regla que falla. Cada regla lleva un comentario que apunta a la regla del back que copia, porque si el back relaja una regla y el front no, el front bloquearía datos válidos.

- **Obligatorio** (líneas 38-40 y los primeros `if` de cada campo): el back usa `NotBlank`, que solo considera vacío el string `''`; unos espacios pasan. El front compara con `''` exactamente, sin `trim()`, para no ser más estricto que el back.
- **Email** (42-47): primero obligatorio; si no está vacío, se comprueba con `emailPattern` (líneas 19-22), el patrón del estándar HTML pero sin exigir dominio de primer nivel, de modo que es al menos tan permisivo como el back.
- **Contraseña** (49-56): mínimo 8 caracteres contados con `[...password.value].length`. El spread recorre el string por caracteres Unicode reales (code points); `.length` a secas cuenta unidades UTF-16, y un emoji contaría como 2. El back (PHP, `mb_strlen`) cuenta code points, así que el front cuenta igual.
- **Confirmación** (58-63): regla solo del front, porque el back nunca recibe la confirmación.

`if / else if` en cada campo hace que, como mucho, haya un mensaje del cliente por campo: si está vacío no tiene sentido decir además que el email no es válido.

El comentario `/** */` de la cabecera (`useRegister.ts:1-10`) registra la decisión de escribir las reglas a mano: el OpenAPI solo declara el mínimo de la contraseña, y una librería de formularios también exigiría copiarlas.

22. submit() paso a paso

`submit` vive en `useForm.ts:73-98` y es el mismo para cualquier formulario. Lo único del registro son los dos huecos que rellena `useRegister`: `action` (paso 5) y el mapa con el 409 (paso 7).

```
async function submit() {
  submitted.value = true                        // 1
  clearGeneralErrors()                          // 2
  if (hasClientErrors()) return null            // 3
  loading.value = true                          // 4
  try {
    return await action()                       // 5
  } catch (err) {
    if (!(err instanceof ApiError)) throw err   // 6
    const handler = errorHandlers[err.code]
    if (handler) {
      handler(err, { setFieldError })           // 7
    } else {
      setFromApiError(err)                      // 8
    }
    return null
  } finally {
    loading.value = false                       // 9
  }
}
```

- 1. A partir de aquí se muestran los errores del cliente (el getter del tema 7 empieza a devolverlos).
- 2. Se borran los errores generales del envío anterior.
- 3. Si alguna regla del cliente falla, no se llama a la API. `hasClientErrors` (`useForm.ts:66-71`) busca algún campo con al menos un mensaje. (Primera ejecución de `clientErrors`, tema 9.)

- 4. El botón pasa a "Creando cuenta..." y se deshabilita.
- 5. Se llama a la `action` que pasó `useRegister`, que llama a `register`. Si va bien, se devuelve el usuario y el componente emite `registered`.
- 6. `client.ts` convierte cualquier fallo de la API en `ApiError`. Si llega otra cosa es un bug del código: se relanza en vez de esconderlo tras un mensaje genérico.
- 7. Si el formulario dio un significado propio a ese código, se ejecuta su función. En el registro, solo `email_already_registered`: el error va al campo email, no arriba. Se busca por `err.code`, nunca por `err.status`: dos errores con el mismo status pueden significar cosas distintas. Desde aquí ese error se comporta como cualquier error del back en un campo: desaparece al editar el email (tema 23).
- 8. Todo lo demás lo resuelve `useFormErrors` (422 por campos, 500, red...).
- 9. `finally` corre siempre, haya ido bien o mal: el botón nunca se queda bloqueado.

23. Qué está mal y cuándo se ve

Dos preguntas distintas, cada una en su composable:

- **Qué está mal** (`useFormErrors`): `fieldErrors` (`useFormErrors.ts:74-87`) es el computed que pinta la plantilla. Para cada campo junta dos listas: los mensajes del cliente que le lleguen y los del back (`backFieldErrors`). Si la lista resultante está vacía, el campo no aparece en el objeto, y por eso la plantilla puede usar `v-if="fieldErrors.email"`.
- **Cuándo se ve** (`useForm`): decide qué errores del cliente le llegan a `useFormErrors` (ninguno hasta el primer envío) y cuándo se vacían los generales (al empezar cada envío).

Las reglas de tiempo siguen el patrón "reward early, punish late" (premia pronto, castiga tarde): no se acusa de un error mientras el usuario aún está escribiendo por primera vez, pero en cuanto corrige algo se le confirma enseguida.

Tipo de error	Aparece	Desaparece	Quién lo decide
Del cliente (reglas)	Al pulsar enviar por primera vez	En vivo, en cuanto el valor cumple la regla	`useForm` (getter con `submitted`)
Del back en un campo (incluido el 409 en el email)	Al recibir la respuesta	Al editar ese campo	`useFormErrors` (`watch`): el error ya no es verdad
General (arriba)	Al recibir la respuesta	En el siguiente envío	`useForm` (`clearGeneralErrors()`)

24. Cronología de una sesión real

Todo lo anterior junto, siguiendo lo que pasa por dentro en un uso real del formulario:

Momento	Qué pasa por dentro	Qué se ve
Abre el formulario	Se crean los refs y computeds de las tres capas. El render lee `fieldErrors`, que llama al getter de `useForm`, que lee `submitted = false`. `clientErrors` no se ejecuta.	Formulario vacío, sin errores
Escribe "ana@" en email	Re-render por `email`. `fieldErrors` no está sucio. `clientErrors` sigue sin ejecutarse.	Nada nuevo

Momento	Qué pasa por dentro	Qué se ve
Pulsa "Crear cuenta"	`submitted = true` (render encolado). `hasClientErrors()`: 1ª ejecución de `clientErrors` → hay errores → `return null`. El render recalcula `fieldErrors`, cuyo getter ahora lee `clientErrors` (caché).	Mensajes debajo de cada campo; el botón sigue habilitado
Completa el email	`email` avisa → `clientErrors` sucio → `fieldErrors` sucio → render encolado → render: se recalculan los dos.	El error del email desaparece al momento
Pulsa con todo bien; el back responde 409	`hasClientErrors()` da `false`. `action` falla con `email_already_registered` → `useForm` encuentra su función en el mapa → `setFieldError('email', ...)`.	Error en email; para ir al login, el enlace "Inicia sesión" del pie
Cambia el email	El `watch` de `useFormErrors` borra el error del back de `email`.	Desaparece el error

25. RegisterForm.vue

El componente queda casi como una plantilla pura: llama al composable, prepara los ids para `aria-describedby` y pinta. No decide nada sobre errores, y no sabe que existen `useForm` ni `useFormErrors`: solo habla con `useRegister`.

- Script (`RegisterForm.vue:1-49`): props (`autofocus`), emits tipados (`switchToLogin`, `registered`), la desestructuración de `useRegister()`, `errorIds` y `describedBy`, y `handleSubmit`, que llama a `submit()` y emite `registered` si devuelve usuario.
- Errores generales arriba (57-68).
- Cada campo: label + input con `:invalid` y `aria-describedby` (o `inputProps` en `Password`) + contenedor de mensajes con el id al que apunta `aria-describedby`.
- Botón de envío: `:disabled="loading"` y texto que cambia mientras carga (232-238).
- Pie: "¿Ya tienes cuenta? Inicia sesión" (240-249), que emite `switchToLogin`.

Desaparecieron `error` y `passwordMismatch`: la contraseña distinta ya no es un caso especial con su propio aviso y su propio bloqueo del botón, sino una regla más de `clientErrors`.

Cuando se partió la lógica en `useForm` y `useFormErrors`, este archivo no cambió: lo que devuelve `useRegister` es lo mismo. Es la ventaja de que el componente dependa de una sola capa.

26. Tests de componente con Vue Test Utils

`src/__tests__/RegisterForm.spec.ts` monta el componente real con `mount()` en un DOM simulado y lo usa como lo haría un usuario: rellena inputs, envía el formulario y comprueba lo que se ve. La API no se llama de verdad.

Piezas que usa

- `vi.mock('../api/auth', ...)` sustituye el módulo entero por uno falso en el que `register` es una función espía (`vi.fn`). Vitest sube (hoist) el `vi.mock` al principio del archivo, así que el componente ya recibe la versión falsa al importarse.
- `vi.mocked(register).mockRejectedValue(new ApiError({...}))` hace que la siguiente llamada falle con ese error. Los errores se construyen con las clases reales, así el test pasa por el mismo `instanceof` que en producción. `mockResolvedValue` es lo contrario: simula éxito.
- `beforeEach(() => vi.mocked(register).mockReset())` deja el espía limpio antes de cada test, para que un test no herede lo que configuró otro.
- `setValue()` escribe en un input y dispara su evento `input`, como teclear; devuelve una promesa que espera a que Vue repinte.
- `trigger('submit.prevent')` + `flushPromises()`: envía el formulario y espera a que terminen todas las promesas pendientes (el `await action()` y el render posterior). `nextTick` solo esperaría un ciclo de render, y aquí hay una promesa de por medio.
- `wrapper.emitted('registered')` devuelve la lista de veces que se emitió ese evento, con sus argumentos, o `undefined` si no se emitió nunca (lo que se comprueba en el 409).

Comprobar errores como lo haría un lector de pantalla

El helper `describedText(wrapper, '#email')` lee el `aria-describedby` del input, busca el elemento con ese id y devuelve su texto. Así el test no solo comprueba que el mensaje "está en la página", sino que está unido a su campo; si se rompiera el enlace ARIA, el test fallaría.

Qué cubren

- Que no hay errores antes del primer envío, aunque los valores sean inválidos.
- Cada regla del cliente: muestra su mensaje y no llama a `register`.
- Que un error del cliente desaparece al corregirlo, y uno del back al editar su campo (y no los de los demás campos).
- 422 por campos y `_global` arriba; 500 con mensaje genérico; error de red con el mensaje de conexión.
- 409: error en email, no se emite `registered`, y el error desaparece al cambiar el email.
- Cambio de comportamiento: el antiguo test "bloquea el envío si las contraseñas no coinciden" se reescribió: ahora el aviso sale al enviar y el botón sigue habilitado.

> **Idea clave: un refactor se demuestra con los tests que NO cambian**
>
> Al introducir `useForm`, este archivo no se tocó y siguió en verde. Como prueba el comportamiento desde fuera (lo que ve el usuario), es la garantía de que repartir la lógica en tres capas no cambió nada visible.

27. Tests de composables con effectScope

`useForm.spec.ts` y `useFormErrors.spec.ts` prueban cada composable **sin montar ningún componente**: crean refs, llaman a la función y comprueban lo que devuelve. Son más rápidos y apuntan mejor a la pieza que falla.

El problema: los `watch` y `computed` de un composable llamado fuera de un componente no tienen dueño, así que nadie los para y seguirían vivos en el test siguiente. La solución es `effectScope()`: un "contenedor" de efectos. Todo lo que se crea dentro de `scope.run(...)` queda ligado a él, y `scope.stop()` lo para todo de golpe.

```
let scope: EffectScope | undefined
afterEach(() => scope?.stop())     // limpia los watch tras cada test

scope = effectScope()
const form = scope.run(() => useForm({ fields, clientErrors, action }))
```

`scope.run` devuelve `undefined` si el scope ya estaba parado; por eso los tests comprueban ese caso y lanzan un error en lugar de usar `!` (que el proyecto evita).

Piezas que usa useForm.spec

- `vi.fn<() => Promise<string>>()` crea una `action` falsa con su tipo, para no depender de `register`: a `useForm` le da igual qué API se llame.
- Para comprobar `loading` **durante** el envío, la `action` devuelve una promesa que el test resuelve a mano: se llama a `submit()` sin `await`, se comprueba que `loading` es `true`, se resuelve la promesa y entonces se espera el resultado.
- `await expect(form.submit()).rejects.toBe(bug)` comprueba que un error que no es `ApiError` se relanza tal cual, y que aun así `loading` vuelve a `false`.
- El caso del mapa pasa un `email_already_registered` con su función y comprueba que el error acaba en el campo email y **no** en los generales (no se delegó).

Qué cubre cada archivo

Archivo	Cubre
`useForm.spec.ts`	No llama a `action` si fallan las reglas; errores del cliente ocultos hasta el primer envío; `loading` durante y después; función del mapa por código; códigos sin función van a `useFormErrors`; relanza lo que no es `ApiError`; limpia los generales en el siguiente envío.
`useFormErrors.spec.ts`	Reparto de `violations` por campo; `_global` y claves desconocidas arriba; mensaje en español por código; mezcla de errores del cliente y del back en el mismo campo; borrar el error del back al editar el campo; `clearGeneralErrors()` no toca los de campo.

El reparto de los tests sigue al del código: los casos de "cuándo se ve" están en `useForm.spec`, y los de "qué está mal" en `useFormErrors.spec`.

28. Tests e2e con Playwright

`e2e/register.spec.ts` abre la app en un navegador real. Aplica aquí porque hay dos cosas que solo un navegador real comprueba: la cabecera `Accept-Language` que sale en la petición y la descripción accesible que calcula el navegador.

- `page.route('**/auth/register', ...)` intercepta la petición antes de que salga. Con `route.fulfill({ status, json })` se responde con exactamente el cuerpo del contrato del back; con `route.abort()` se simula que no hay red.

- `route.request().headers()['accept-language']` captura la cabecera para comprobar después que vale `es`.
- `getByRole` / `getByLabel` buscan elementos como los encuentra un usuario o un lector de pantalla (por su rol y su etiqueta), no por clases CSS.
- `toHaveAccessibleDescription(...)` pide al navegador la descripción accesible del input, la que construye a partir de `aria-describedby`. Es la versión "de verdad" del `describedText` de los tests unitarios. El test del 409 solo necesita esto: comprueba que el email queda descrito con "Ya existe una cuenta registrada con ese email.".
- Los helpers `openRegisterForm` y `fillAndSubmit` evitan repetir los pasos en cada test; `fillAndSubmit` usa valores que pasan las reglas del cliente, para que la petición llegue al back simulado.

Anexo · Chuleta

Concepto	En una línea
`ref(x)`	Caja reactiva; leer `.value` suscribe, escribir `.value` avisa.
`shallowRef(x)`	Solo avisa al reemplazar `.value` entero; con primitivos, igual que `ref`.
`computed(fn)`	Valor derivado y cacheado; sin efectos dentro.
Dependencias	Se apuntan al ejecutar; solo cuenta lo que se leyó en esa ejecución.
Perezoso	Un computed avisado solo se marca sucio; se recalcula cuando alguien lo lee.
Render encolado	El repintado ocurre al terminar el código síncrono, una vez por tanda de cambios.
`watch(src, cb)`	Efecto cuando cambia `src`; por defecto corre antes del render.
`readonly(r)`	Vista de solo lectura: el consumidor lee, solo el dueño escribe.
Composable	Función `useX` con estado reactivo; cada llamada, estado propio.
Composables en capas	Cada capa una responsabilidad; cada una solo habla con la de al lado.
`MaybeRefOrGetter` + `toValue`	Aceptar valor, ref o getter; `toValue` dentro de un computed mantiene la reactividad.
Getter dentro de un computed	Sin caché propia; sus lecturas cuentan como dependencias del computed que lo llama.
`Partial<Record<K, V>>`	Objeto que puede tener algunas de las claves `K`.
Genéricos inferidos	`useForm({ fields, action })`: `Field` y `Result` salen de los argumentos.
`export type { X }`	Reexportar un tipo como parte de tu API; se borra al compilar.
Inversión de control	Pasas funciones (`action`, mapa por código) y el composable decide cuándo llamarlas.
`v-for` + `:key`	Repetir; la key identifica cada elemento.
`:attr="undefined"`	Vue quita el atributo del DOM.
`defineEmits`	El hijo avisa, el padre decide.
Fallthrough	Atributos no declarados caen en la raíz; si el input va envuelto, usa `inputProps`.
`novalidate`	Sin burbujas del navegador; mantén `required` por accesibilidad.
`aria-invalid` / `aria-describedby`	Campo no válido / qué texto lo describe.
Botón y validación	Deshabilitar solo mientras carga, nunca por errores.
Reward early, punish late	Errores tras el primer envío; se quitan en cuanto se corrigen. Lo decide `useForm`.
Qué está mal / cuándo se ve	`useFormErrors` / `useForm`.
`err.code`, no `err.status`	Decide por el significado, no por el número HTTP.
`vi.mock` + `mockRejectedValue`	API falsa que falla con el error que quieras.
`flushPromises()`	Esperar a todas las promesas pendientes, no solo a un render.
`effectScope()`	Probar un composable sin componente; `scope.stop()` para sus `watch`.
`page.route` + `fulfill`/`abort`	Back simulado en el navegador real.

Concepto	En una línea
`toHaveAccessibleDescription`	Comprueba el mensaje unido al campo, como lo lee un lector de pantalla.